

Assignment - 2

1)

1.1)

$$p(\theta|y) = \frac{\mathcal{L}(\theta|y) p(\theta)}{\int \mathcal{L}(\theta|y) p(\theta) d\theta}$$

$$= \frac{\frac{10!}{7! \times 3!} \theta^7 (1-\theta)^3 (1)}{\frac{1}{11}}$$

$$= \left(\frac{11 \times 10!}{7! \times 3!} \right) \theta^7 (1-\theta)^3$$

$$= 1320 \theta^7 (1-\theta)^3$$

(a) $\theta = 0.75$

$$\begin{aligned} p(0.75|y) &= 1320 \times (0.75)^7 \times (0.25)^3 \\ &= 2.752 \end{aligned}$$

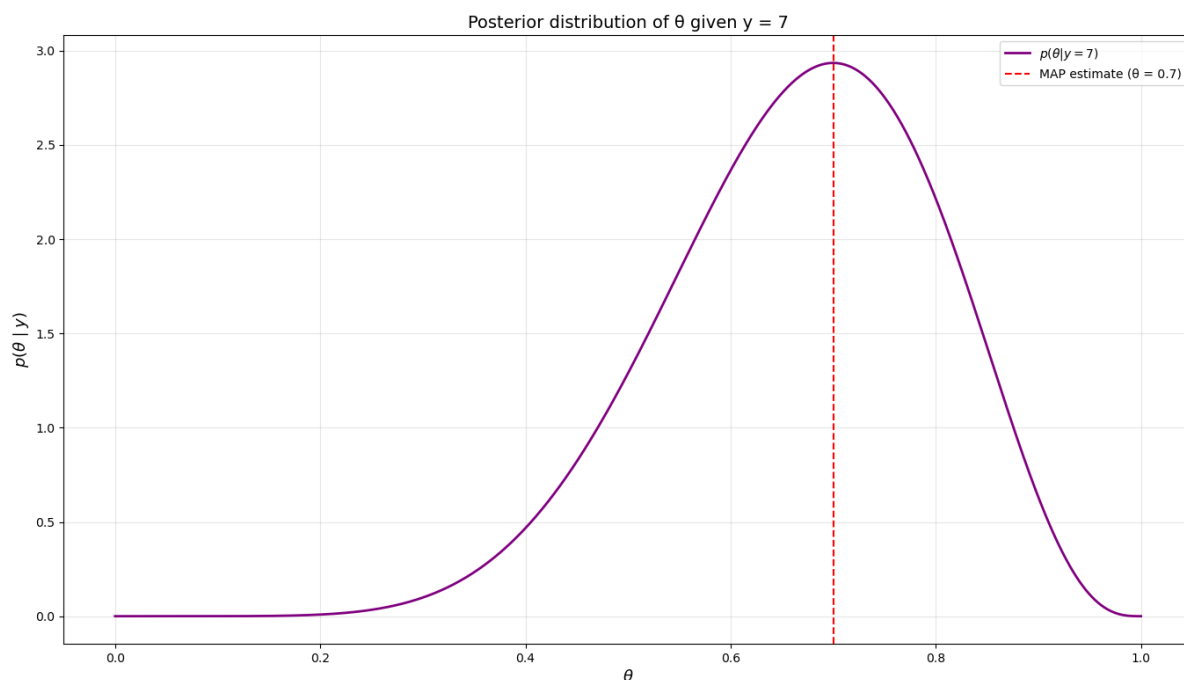
(b) $\theta = 0.25$

$$\begin{aligned} p(0.25|y) &= 1320 \times (0.25)^7 \times (0.25)^3 \\ &= 0.034 \end{aligned}$$

(c) $\theta = 1$

$$p(1|y) = 1320 \times (1)^7 (0)^3 = 0$$

1.2)



1.3) $p(\theta|y) = 1320 \theta^7 (1-\theta)^3$

so, for maximizing this
lets take natural log since it is an increasing function

$$\ln(1320) + 7 \ln \theta + 3 \ln(1-\theta)$$

then, differentiate it wrt θ and equate to 0.

$$= 0 + \frac{7}{\theta} + \frac{3}{(1-\theta)} (-1) = 0$$

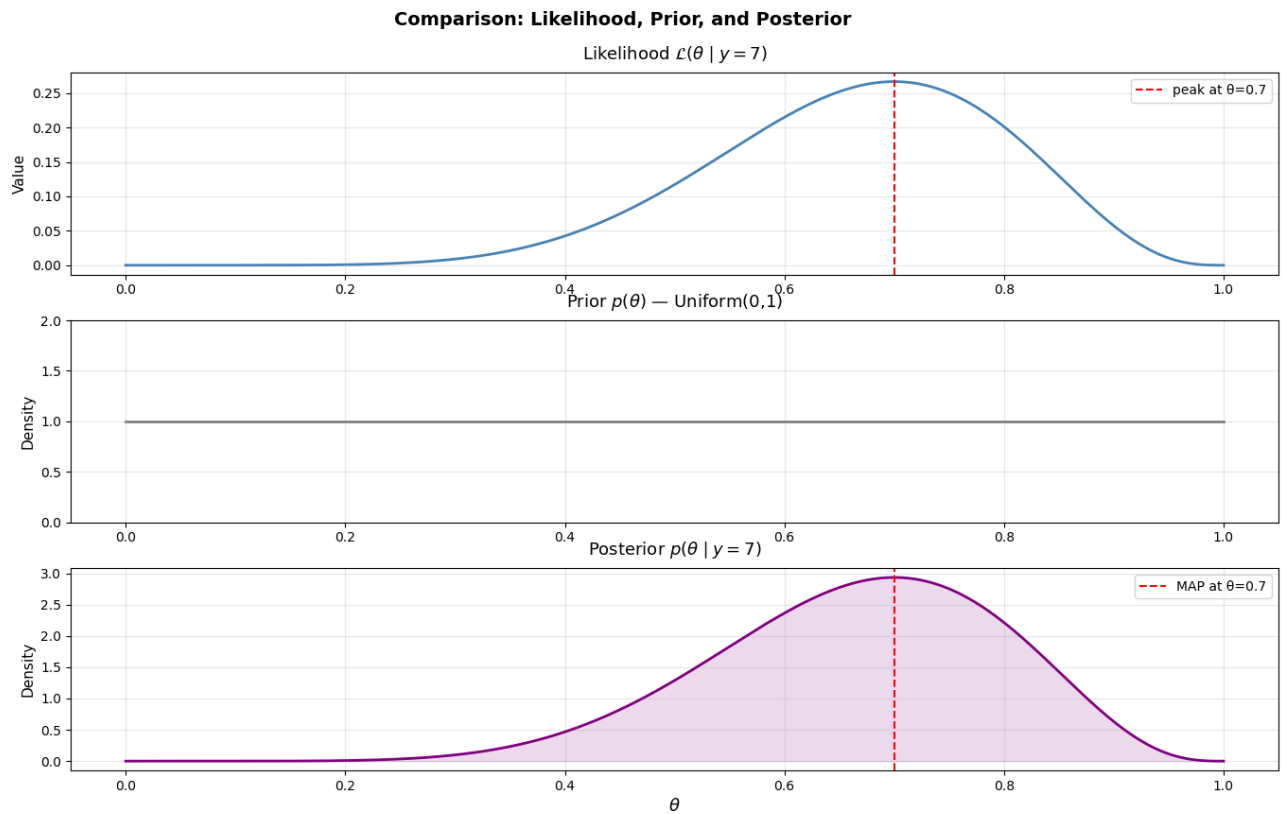
$$\Rightarrow \frac{7}{\theta} = \frac{3}{1-\theta}$$

$$7 - 7\theta = 3\theta$$

$$10\theta = 7$$

$$\theta = \frac{7}{10} = 0.7$$

1.4)



$$2) \quad 2.1) \quad p'(\mu | \sigma, y) = \mathcal{L}(\mu, \sigma | y) p(\mu)$$

$$= \frac{1}{(\sigma\sqrt{2\pi})^n} e^{-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2} \times \frac{1}{25\sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{\mu - 250}{25} \right)^2}$$

$$= \frac{1}{(50\sqrt{2\pi})^8} e^{-\frac{1}{2(50)^2} \sum_{i=1}^8 (y_i - \mu)^2} \times \frac{1}{25\sqrt{2\pi}} e^{-\frac{1}{2} (25)^2 (\mu - 250)^2}$$

for $\mu = 300$

$$\sum (y_i - 300)^2 = 0 + 900 + 8100 + 22500 + 40000 + 100 + 144400 + 22500 = 238500$$

$$= \frac{1}{(50\sqrt{2\pi})^8} e^{-\frac{238500}{5000}} \left(\frac{1}{25\sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{300 - 250}{25} \right)^2} \right)$$

$$= 1.91 \times 10^{-22}$$

for $\mu = 900$

$$\sum (y_i - 900)^2 = 360000 + 396900 + 260100 + 202500 + 160000 + 372100 + 48400 + 202500$$

$$= \frac{1}{(50\sqrt{2\pi})^8} e^{-\frac{2002500}{5000}} \left(\frac{1}{25\sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{650}{25}\right)^2} \right)$$

≈ 0

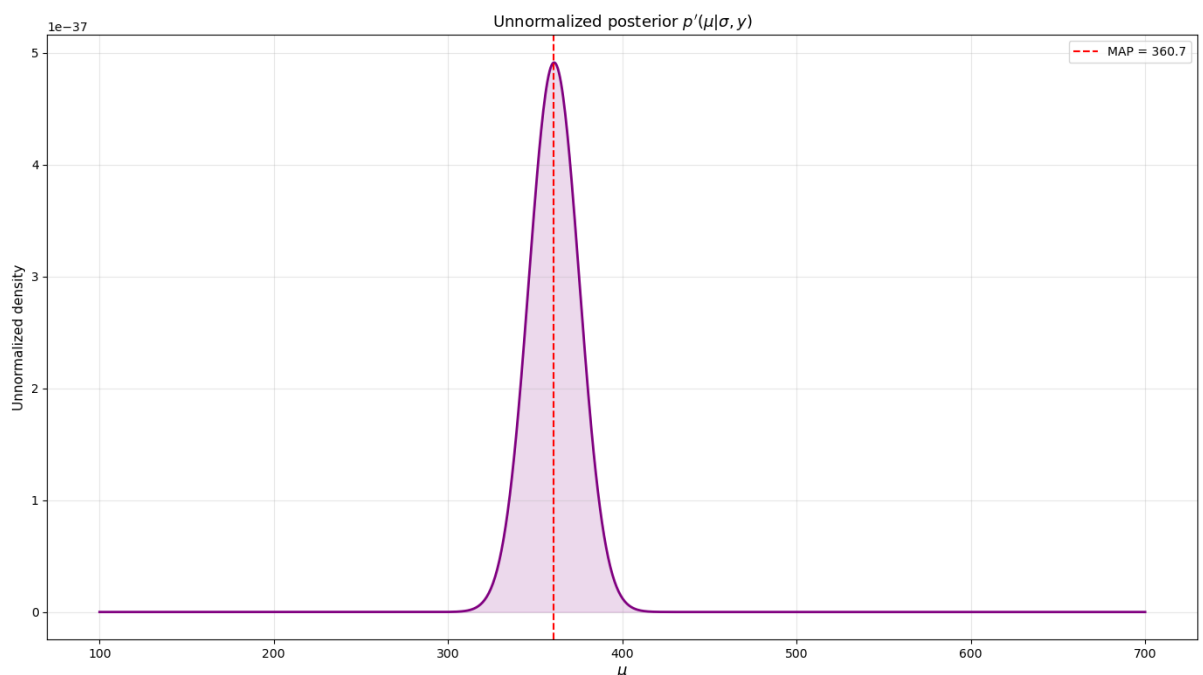
for $\mu = 50$

$$\sum (y_i - 50)^2 = 62500 + 48400 + 115600 + 160000 + 202500 + 57600 + 390400 + 160000 = 1197000$$

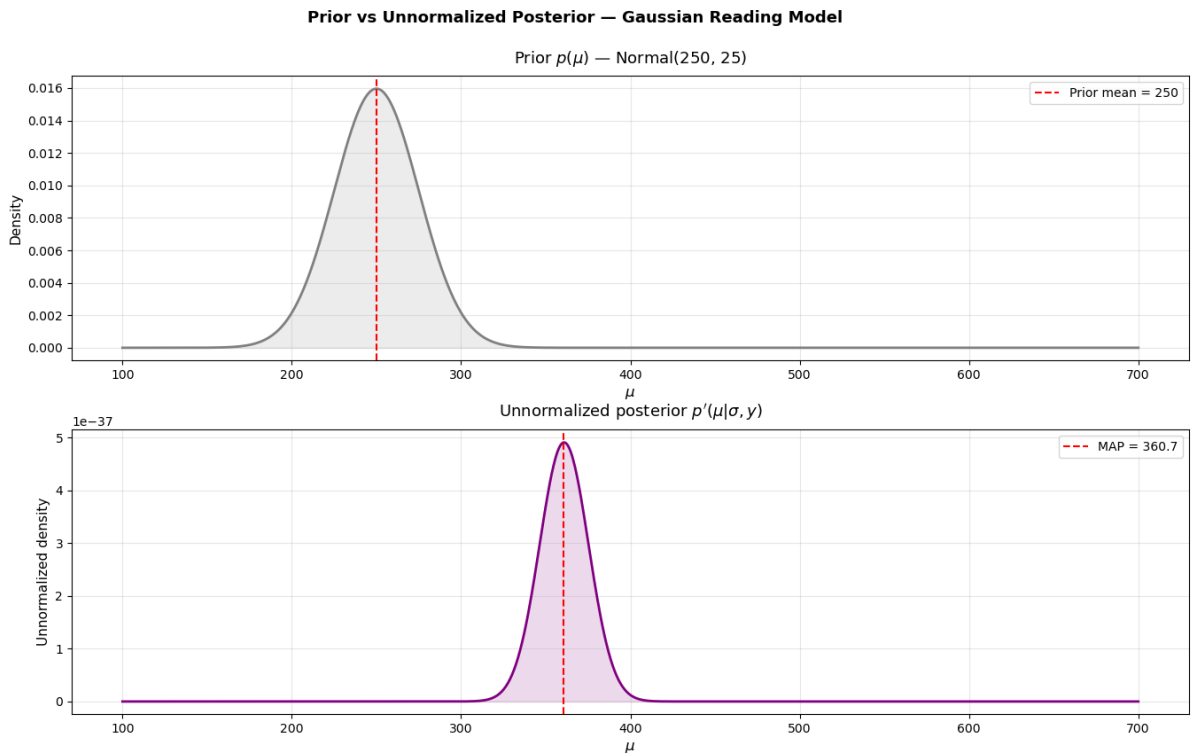
$$= \frac{1}{(50\sqrt{2\pi})^8} e^{-\frac{1197000}{5000}} \left(\frac{1}{25\sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{200}{25}\right)^2} \right)$$

$\approx 0.$

2.2)



2.3)



3) 3.1) So, the posterior is Gamma with $\alpha = \text{previous } \alpha + \text{data}$
and $\beta = \text{previous } \beta + \text{no. of observations}$

Day	Data, k	Prior	Posterior
1	25	Gamma(40, 2)	Gamma(65, 3)
2	20	Gamma(65, 3)	Gamma(85, 4)
3	23	Gamma(85, 4)	Gamma(108, 5)
4	27	Gamma(108, 5)	Gamma(135, 6)

So, prior for day 5 is $\lambda \sim \text{Gamma}(135, 6)$

3.2) Predicted number of road accidents is

$$E(K_5) = E(\lambda)$$

because

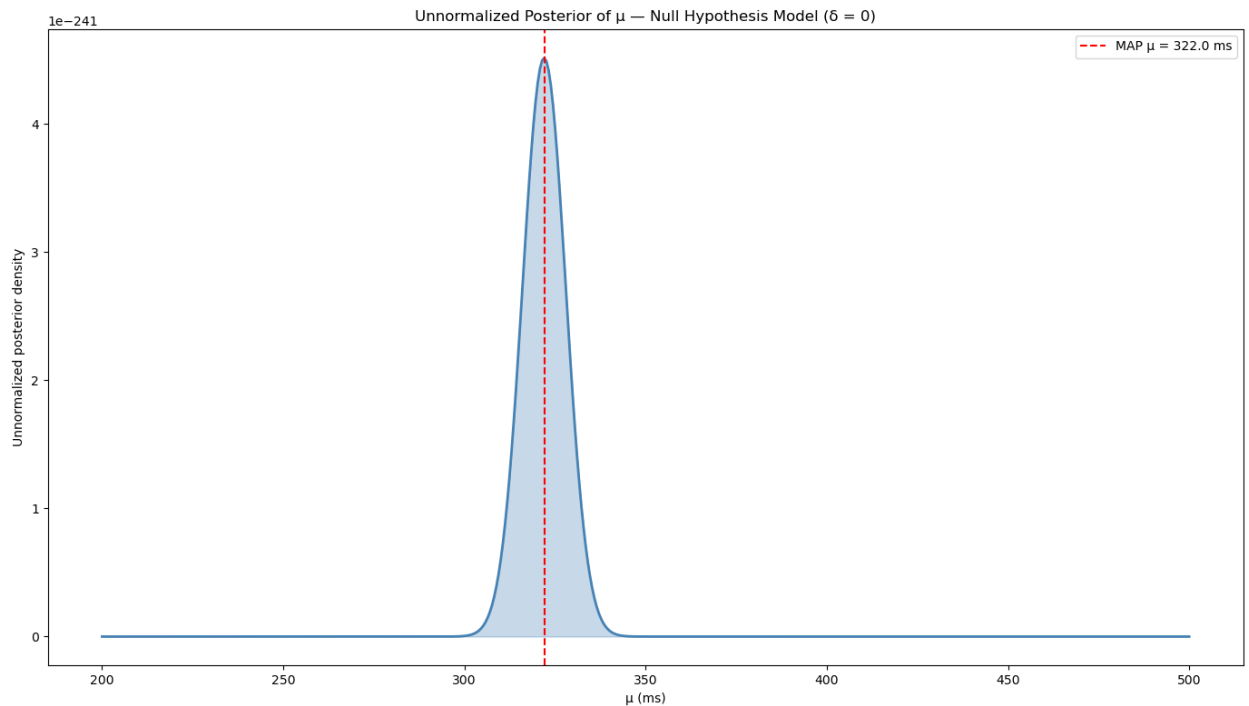
$$E(K_5 | \lambda) = \lambda$$

$$E(\lambda) = \frac{\alpha}{\beta} = \frac{135}{6} = 22.5$$

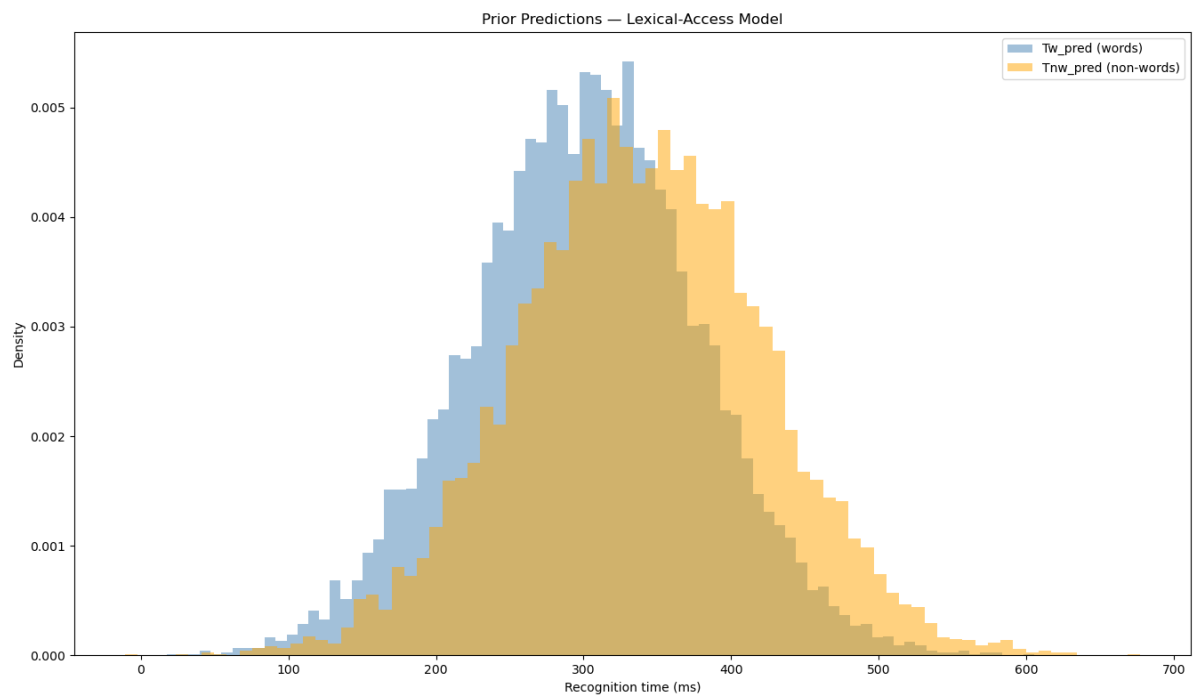
Suggesting that 22 or 23 accidents are predicted to happen on day 5.

4.5)

4.5.1)



4.5.2)



for 10000 samples of μ and δ

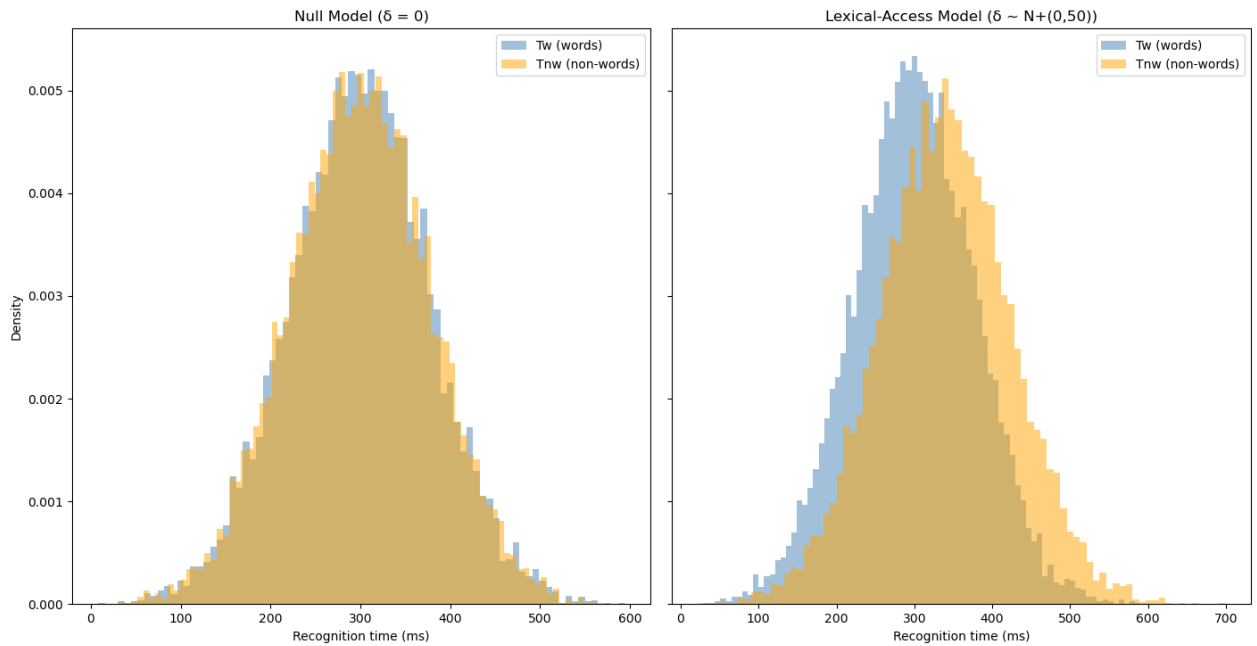
Mean Tw_pred = 300.78 ms

Mean Tnw_pred = 339.32 ms

Expected shift due to delta prior mean (mean of delta samples) = 40.31 ms

4.5.3)

Prior Predictions: Null vs Lexical-Access Model



Null model

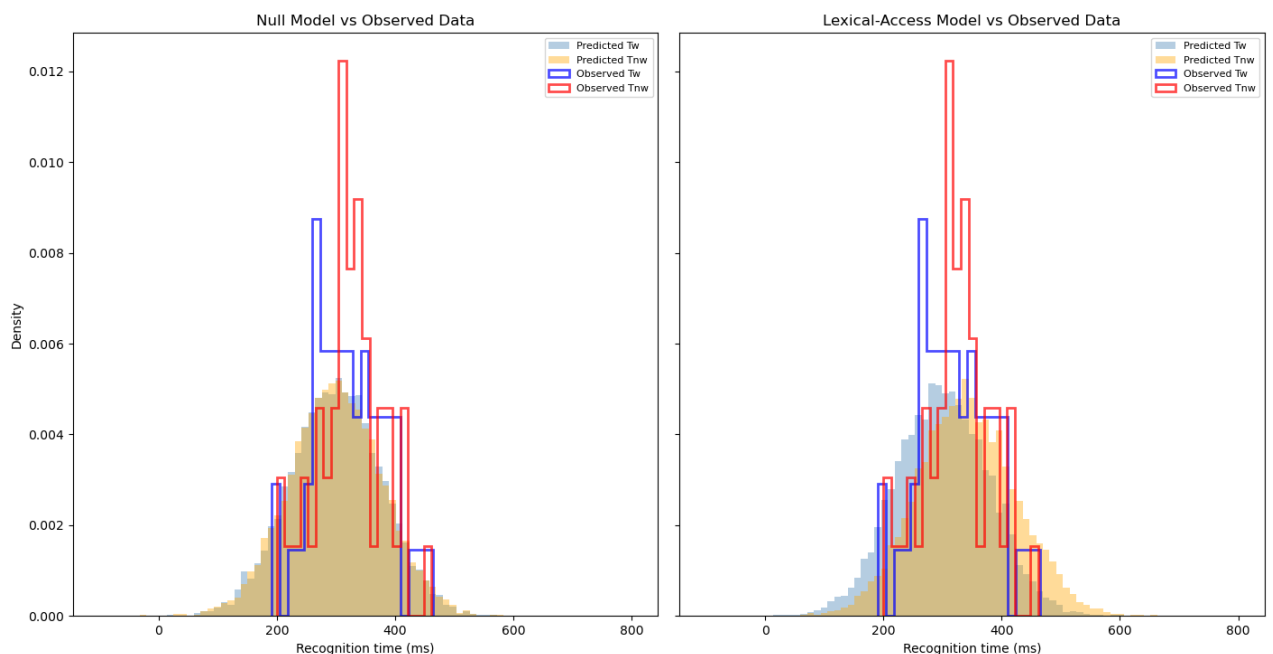
Lexical-access model

Mean $T_w = 299.82$ ms
 Mean $T_{nw} = 299.65$ ms
 Difference = -0.17 ms

Mean $T_w = 299.90$ ms
 Mean $T_{nw} = 339.82$ ms
 Difference = 39.92 ms

4.5.4)

Prior Predictions vs Observed Data



Observed data

Mean $T_w = 321.37$ ms

Mean $T_{rw} = 323.24$ ms

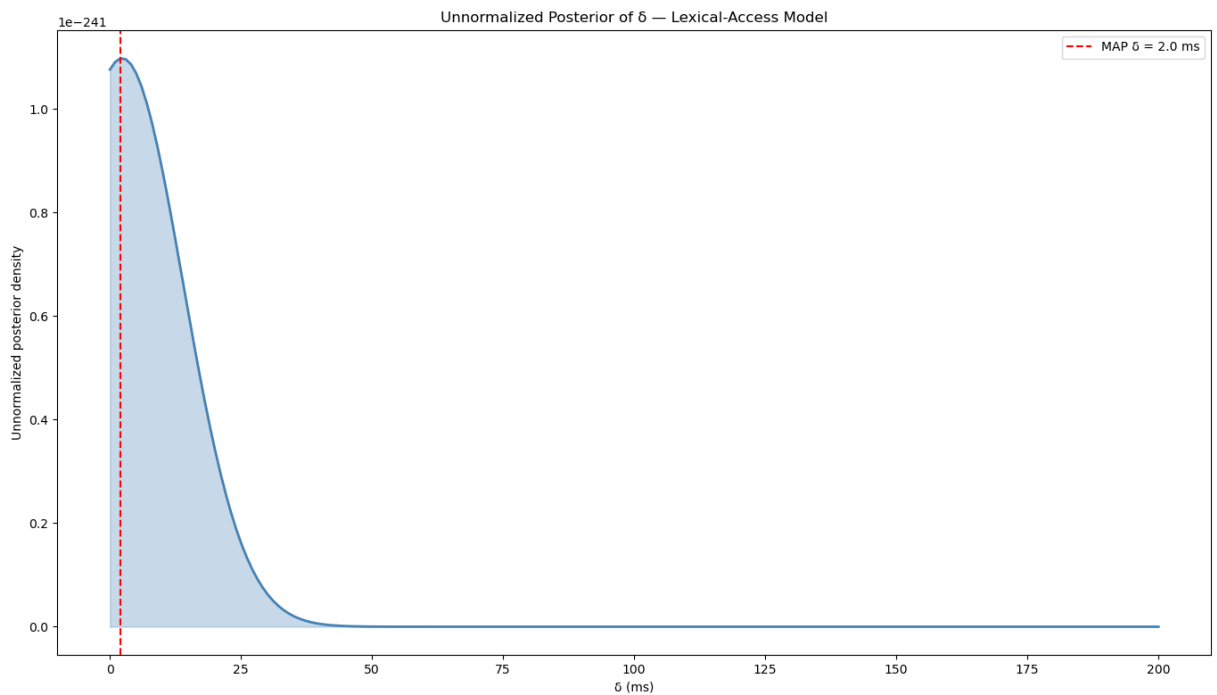
Observed difference = 1.86 ms

Lexical-access prior predicted difference ≈ 40 ms

Null prior predicted difference ≈ 0 ms

So, Null hypothesis model seems more consistent since 1.86 is closer to 0 than 40.

4.5.5)




```

import numpy as np
import matplotlib.pyplot as plt
from math import comb
from scipy.stats import norm
from scipy.stats import truncnorm
import pandas as pd
y = 7
n = 10
marginal_likelihood = 1/11
theta = np.linspace(0, 1, 1000)
coeff = comb(n, y)
posterior = (coeff * theta**y * (1 - theta)**(n - y)) / marginal_likelihood
plt.figure(figsize=(8, 5))
plt.plot(theta, posterior, color='purple', linewidth=2, label=r'$p(\theta|y=7)$')
plt.axvline(x=0.7, color='red', linestyle='--', linewidth=1.5, label='MAP
    estimate ( $\theta = 0.7$ )')
plt.xlabel(r'$\theta$', fontsize=13)
plt.ylabel(r'$p(\theta \mid y=7)$', fontsize=13)
plt.title('Posterior distribution of  $\theta$  given  $y = 7$ ', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

likelihood = coeff * theta**y * (1 - theta)**(n - y)
prior = np.ones_like(theta)
posterior = likelihood / marginal_likelihood

fig, axes = plt.subplots(3, 1, figsize=(8, 12))

axes[0].plot(theta, likelihood, color='steelblue', linewidth=2)
axes[0].axvline(x=0.7, color='red', linestyle='--', linewidth=1.5, label='peak at
     $\theta=0.7$ ')
axes[0].set_title(r'Likelihood  $\mathcal{L}(\theta \mid y=7)$ ', fontsize=13, pad
    =10)
axes[0].set_ylabel('Value', fontsize=11)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(theta, prior, color='gray', linewidth=2)
axes[1].set_title(r'Prior  $p(\theta) \sim \text{Uniform}(0,1)$ ', fontsize=13, pad=10)
axes[1].set_ylabel('Density', fontsize=11)
axes[1].set_ylim(0, 2)
axes[1].grid(True, alpha=0.3)

axes[2].plot(theta, posterior, color='purple', linewidth=2)
axes[2].fill_between(theta, posterior, alpha=0.15, color='purple')
axes[2].axvline(x=0.7, color='red', linestyle='--', linewidth=1.5, label='MAP at
     $\theta=0.7$ ')
axes[2].set_title(r'Posterior  $p(\theta \mid y=7)$ ', fontsize=13, pad=10)
axes[2].set_ylabel('Density', fontsize=11)
axes[2].set_xlabel(r'$\theta$', fontsize=13)
axes[2].legend()
axes[2].grid(True, alpha=0.3)

```

```

plt.suptitle('Comparison: Likelihood, Prior, and Posterior', fontsize=14,
             fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()

y = np.array([300, 270, 390, 450, 500, 290, 680, 450])
n = len(y)
sigma = 50
prior_mean = 250
prior_sd = 25

mu = np.linspace(100, 700, 1000)

likelihood = np.array([
    (1 / (sigma * np.sqrt(2 * np.pi)))**n * np.exp(-np.sum((y - m)**2) / (2 *
    sigma**2))
    for m in mu
])

prior_vals = norm.pdf(mu, loc=prior_mean, scale=prior_sd)

unnorm_posterior = likelihood * prior_vals

map_mu = mu[np.argmax(unnorm_posterior)]

plt.figure(figsize=(8, 5))
plt.plot(mu, unnorm_posterior, color='purple', linewidth=2)
plt.fill_between(mu, unnorm_posterior, alpha=0.15, color='purple')
plt.axvline(x=map_mu, color='red', linestyle='--', linewidth=1.5, label=f'MAP = {
    map_mu:.1f}')
plt.xlabel(r'$\mu$', fontsize=13)
plt.ylabel("Unnormalized density", fontsize=11)
plt.title(r"Unnormalized posterior $p(\mu|\sigma,y)$", fontsize=13)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(2, 1, figsize=(8, 10))

axes[0].plot(mu, prior_vals, color='gray', linewidth=2)
axes[0].fill_between(mu, prior_vals, alpha=0.15, color='gray')
axes[0].axvline(x=prior_mean, color='red', linestyle='--', linewidth=1.5, label='
    Prior mean = 250')
axes[0].set_title(r'Prior $p(\mu)$ Normal(250, 25)', fontsize=13, pad=10)
axes[0].set_ylabel('Density', fontsize=11)
axes[0].set_xlabel(r'$\mu$', fontsize=12)
axes[0].legend()

```

```

axes[0].grid(True, alpha=0.3)

axes[1].plot(mu, unnorm_posterior, color='purple', linewidth=2)
axes[1].fill_between(mu, unnorm_posterior, alpha=0.15, color='purple')
axes[1].axvline(x=map_mu, color='red', linestyle='--', linewidth=1.5, label=f'MAP
= {map_mu:.1f}')
axes[1].set_title(r"Unnormalized posterior  $p(\mu|\sigma,y)$ ", fontsize=13, pad
=10)
axes[1].set_ylabel('Unnormalized density', fontsize=11)
axes[1].set_xlabel(r' $\mu$ ', fontsize=12)
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Prior vs Unnormalized Posterior Gaussian Reading Model', fontsize
=13, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()

url = "https://raw.githubusercontent.com/yadavhimanshu059/CGS698C/main/notes/
Module-2/recognition.csv"
dat = pd.read_csv(url, index_col=0)
Tw = dat["Tw"].values
Tnw = dat["Tnw"].values

sigma = 60
delta = 0

mu_grid = np.linspace(200, 500, 500)

unnorm_posterior = []

for mu in mu_grid:
    L_Tw = np.prod(norm.pdf(Tw, loc=mu, scale=sigma))
    L_Tnw = np.prod(norm.pdf(Tnw, loc=mu + delta, scale=sigma))
    p_mu = norm.pdf(mu, loc=300, scale=50)

    unnorm_posterior.append(L_Tw * L_Tnw * p_mu)

unnorm_posterior = np.array(unnorm_posterior)

map_mu = mu_grid[np.argmax(unnorm_posterior)]
print(f"MAP estimate of mu: {map_mu:.2f} ms")

plt.figure(figsize=(8, 5))
plt.plot(mu_grid, unnorm_posterior, color="steelblue", linewidth=2)
plt.fill_between(mu_grid, unnorm_posterior, alpha=0.3, color="steelblue")
plt.axvline(map_mu, color="red", linestyle="--", label=f'MAP  $\mu = \{map\_mu:.1f\}$  ms
")
plt.xlabel(" $\mu$  (ms)")
plt.ylabel("Unnormalized posterior density")

```

```

plt.title("Unnormalized Posterior of  $\mu$  Null Hypothesis Model ( $\delta = 0$ )")
plt.legend()
plt.tight_layout()
plt.show()

N = 10000

mu_samples = np.random.normal(loc=300, scale=50, size=N)

a = (0 - 0) / 50
delta_samples = truncnorm.rvs(a=a, b=np.inf, loc=0, scale=50, size=N)

Tw_pred = np.random.normal(loc=mu_samples, scale=60)
Tnw_pred = np.random.normal(loc=mu_samples+delta_samples, scale=60)

plt.figure(figsize=(8, 5))
plt.hist(Tw_pred, bins=80, alpha=0.5, color="steelblue", label="Tw_pred (words)",
         density=True)
plt.hist(Tnw_pred, bins=80, alpha=0.5, color="orange", label="Tnw_pred (non-words)", density=True)
plt.xlabel("Recognition time (ms)")
plt.ylabel("Density")
plt.title("Prior Predictions Lexical-Access Model")
plt.legend()
plt.tight_layout()
plt.show()

print(f"Mean Tw_pred: {Tw_pred.mean():.2f} ms")
print(f"Mean Tnw_pred: {Tnw_pred.mean():.2f} ms")
print(f"Expected shift due to delta prior mean: {delta_samples.mean():.2f} ms")

N = 10000
sigma = 60

mu_null = np.random.normal(300, 50, N)
Tw_null = np.random.normal(mu_null, sigma)
Tnw_null = np.random.normal(mu_null + 0, sigma) # delta=0

mu_lex = np.random.normal(300, 50, N)
a = (0 - 0) / 50
delta_lex = truncnorm.rvs(a=a, b=np.inf, loc=0, scale=50, size=N)
Tw_lex = np.random.normal(mu_lex, sigma)
Tnw_lex = np.random.normal(mu_lex + delta_lex, sigma)

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

axes[0].hist(Tw_null, bins=80, alpha=0.5, color="steelblue", label="Tw (words)",
             density=True)

```

```

axes[0].hist(Tnw_null, bins=80, alpha=0.5, color="orange", label="Tnw (non-
words)", density=True)
axes[0].set_title("Null Model ( $\delta = 0$ )")
axes[0].set_xlabel("Recognition time (ms)")
axes[0].set_ylabel("Density")
axes[0].legend()

axes[1].hist(Tw_lex, bins=80, alpha=0.5, color="steelblue", label="Tw (words)",
density=True)
axes[1].hist(Tnw_lex, bins=80, alpha=0.5, color="orange", label="Tnw (non-
words)", density=True)
axes[1].set_title("Lexical-Access Model ( $\delta \sim N(0,50)$ )")
axes[1].set_xlabel("Recognition time (ms)")
axes[1].legend()

plt.suptitle("Prior Predictions: Null vs Lexical-Access Model", fontsize=13)
plt.tight_layout()
plt.show()

print("Null model:")
print(f" Mean Tw: {Tw_null.mean():.2f} ms")
print(f" Mean Tnw: {Tnw_null.mean():.2f} ms")
print(f" Difference: {Tnw_null.mean() - Tw_null.mean():.2f} ms")

print("\nLexical-access model:")
print(f" Mean Tw: {Tw_lex.mean():.2f} ms")
print(f" Mean Tnw: {Tnw_lex.mean():.2f} ms")
print(f" Difference: {Tnw_lex.mean() - Tw_lex.mean():.2f} ms")

url = "https://raw.githubusercontent.com/yadavhimanshu059/CGS698C/main/notes/
Module-2/recognition.csv"
dat = pd.read_csv(url, index_col=0)
Tw = dat["Tw"].values
Tnw = dat["Tnw"].values

N = 10000
sigma = 60

mu_null = np.random.normal(300, 50, N)
Tw_null = np.random.normal(mu_null, sigma)
Tnw_null = np.random.normal(mu_null, sigma)

mu_lex = np.random.normal(300, 50, N)
a = (0 - 0) / 50
delta_lex = truncnorm.rvs(a=a, b=np.inf, loc=0, scale=50, size=N)
Tw_lex = np.random.normal(mu_lex, sigma)
Tnw_lex = np.random.normal(mu_lex + delta_lex, sigma)

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

bins = np.linspace(-100, 800, 80)

```

```

axes[0].hist(Tw_null, bins=bins, alpha=0.4, color="steelblue", label="Predicted
Tw", density=True)
axes[0].hist(Tnw_null, bins=bins, alpha=0.4, color="orange", label="Predicted
Tnw", density=True)
axes[0].hist(Tw, bins=20, alpha=0.7, color="blue", label="Observed
Tw", density=True, histtype="step", linewidth=2)
axes[0].hist(Tnw, bins=20, alpha=0.7, color="red", label="Observed
Tnw", density=True, histtype="step", linewidth=2)
axes[0].set_title("Null Model vs Observed Data")
axes[0].set_xlabel("Recognition time (ms)")
axes[0].set_ylabel("Density")
axes[0].legend(fontsize=8)

axes[1].hist(Tw_lex, bins=bins, alpha=0.4, color="steelblue", label="Predicted
Tw", density=True)
axes[1].hist(Tnw_lex, bins=bins, alpha=0.4, color="orange", label="Predicted
Tnw", density=True)
axes[1].hist(Tw, bins=20, alpha=0.7, color="blue", label="Observed
Tw", density=True, histtype="step", linewidth=2)
axes[1].hist(Tnw, bins=20, alpha=0.7, color="red", label="Observed
Tnw", density=True, histtype="step", linewidth=2)
axes[1].set_title("Lexical-Access Model vs Observed Data")
axes[1].set_xlabel("Recognition time (ms)")
axes[1].legend(fontsize=8)

plt.suptitle("Prior Predictions vs Observed Data", fontsize=13)
plt.tight_layout()
plt.show()

print("Observed data:")
print(f" Mean Tw: {Tw.mean():.2f} ms")
print(f" Mean Tnw: {Tnw.mean():.2f} ms")
print(f" Observed difference: {Tnw.mean() - Tw.mean():.2f} ms")

print("\nLexical-access prior predicted difference: ~40 ms")
print("Null prior predicted difference: ~0 ms")
print(f"\nObserved difference ({Tnw.mean() - Tw.mean():.2f} ms) is much closer to
Null model's prediction (0 ms)")

url = "https://raw.githubusercontent.com/yadavhimanshu059/CGS698C/main/notes/
Module-2/recognition.csv"
dat = pd.read_csv(url, index_col=0)
Tw = dat["Tw"].values
Tnw = dat["Tnw"].values

sigma = 60

delta_grid = np.linspace(0, 200, 200)
mu_grid = np.linspace(100, 600, 150)
dmu = mu_grid[1] - mu_grid[0]

```

```

a = (0 - 0) / 50

post_delta = []

for delta in delta_grid:

    integral_mu = 0.0
    for mu in mu_grid:
        L_Tw      = np.prod(norm.pdf(Tw, loc=mu, scale=sigma))
        L_Tnw     = np.prod(norm.pdf(Tnw, loc=mu + delta, scale=sigma))
        p_mu      = norm.pdf(mu, loc=300, scale=50)
        integral_mu += L_Tw * L_Tnw * p_mu * dmu

    p_delta = truncnorm.pdf(delta, a=a, b=np.inf, loc=0, scale=50)

    post_delta.append(integral_mu * p_delta)

post_delta = np.array(post_delta)

map_delta = delta_grid[np.argmax(post_delta)]
print(f"MAP estimate of delta: {map_delta:.2f} ms")

plt.figure(figsize=(8, 5))
plt.plot(delta_grid, post_delta, color="steelblue", linewidth=2)
plt.fill_between(delta_grid, post_delta, alpha=0.3, color="steelblue")
plt.axvline(map_delta, color="red", linestyle="--", label=f"MAP  $\delta$  = {map_delta:.1f} ms")
plt.xlabel(" $\delta$  (ms)")
plt.ylabel("Unnormalized posterior density")
plt.title("Unnormalized Posterior of  $\delta$  Lexical-Access Model")
plt.legend()
plt.tight_layout()
plt.show()

```